



Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н. Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ _____ «СПЕЦИАЛЬНОЕ МАШИНОСТРОЕНИЕ» _____

КАФЕДРА _____ «РАКЕТНЫЕ И ИМПУЛЬСНЫЕ СИСТЕМЫ» (СМ-6) _____

Лабораторная работа № 1

ПО ДИСЦИПЛИНЕ:

Компьютерное моделирование

НА ТЕМУ:

«Клеточные автоматы. Двумерная модель дорожного движения
BML»

Выполнили: студенты
группы

СМ6-92

(подпись, дата)

Р.Д. Кучеренко
(И.О. Фамилия)

(подпись, дата)

А.Н. Крикоров
(И.О. Фамилия)

Проверил

(подпись, дата)

В.А. Федулов
(И.О. Фамилия)

Москва, 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Теоретические сведения	5
2 Постановка задачи и формализация модели	8
3 Программная реализация.....	10
4 Вычислительный эксперимент	13
5 Верификация и обсуждение результатов.....	19
ЗАКЛЮЧЕНИЕ	21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	22
ПРИЛОЖЕНИЕ А. Листинг ключевых функций	23

ВВЕДЕНИЕ

Клеточные автоматы представляют собой дискретные модели, в которых пространство разделено на регулярную совокупность клеток, время изменяется дискретными тактами, а состояние каждой клетки определяется локальным правилом перехода. Несмотря на простоту локальных правил, такие системы способны демонстрировать сложное коллективное поведение, включая самоорганизацию, образование устойчивых структур и фазовые переходы [1].

В данной лабораторной работе рассматривается двумерная клеточная модель дорожного движения Бихама–Миддлтона–Левина (BML). Модель предназначена для исследования возникновения заторов в системе двух взаимно перпендикулярных транспортных потоков. Основное внимание уделяется зависимости средней скорости автомобилей от плотности заполнения поля и переходу от свободного движения к блокировке.

Программная реализация выполнена на языке Python с применением библиотек NumPy и Matplotlib. NumPy используется для хранения и векторной обработки клеточного поля, Matplotlib — для построения графиков и анимации [2, 3].

Ссылка на открытый репозиторий с исходным кодом:
<https://github.com/RII01/1-lab-BPL>

Цель работы

Разработать и исследовать двумерную клеточную модель дорожного движения BML, определить влияние плотности автомобилей на среднюю скорость и транспортный поток, а также визуально проследить формирование заторов.

Задачи работы

- изучить основные понятия и свойства клеточных автоматов;
- формализовать двумерную модель дорожного движения BML;
- реализовать алгоритм синхронного перемещения автомобилей двух направлений;
- создать визуализацию эволюции системы и автоматическое сохранение контрольных состояний;
- провести серию вычислительных экспериментов при различных значениях плотности;
- построить зависимости средней скорости и транспортного потока от плотности автомобилей;
- выполнить верификацию программной реализации и интерпретировать полученные результаты.

1 Теоретические сведения

1.1 Клеточные автоматы

Классический клеточный автомат — дискретная динамическая система, состоящая из регулярной решётки клеток. Каждая клетка в каждый момент времени находится ровно в одном состоянии из конечного множества. Состояния всех клеток изменяются синхронно на каждом такте по локальному однородному правилу, зависящему от состояния самой клетки и её окрестности [1].

Формально клеточный автомат можно представить набором $\langle A, S, N, F \rangle$, где A — дискретное клеточное пространство, S — конечное множество состояний клетки, N — выбранная окрестность, F — локальная функция перехода. Время t принимает дискретные значения $0, 1, 2, \dots$.

1.2 Граничные условия и синхронное обновление

В клеточных автоматах состояние граничных клеток требует специального определения. В учебных материалах рассматриваются периодические, фиксированные, адиабатические и отражающие граничные условия [1]. В модели BML используются периодические границы: выход автомобиля за правую границу приводит к его появлению в крайнем левом столбце той же строки, а выход за нижнюю границу — к появлению в верхней строке того же столбца.

Синхронность обновления принципиальна: решение о перемещении всех автомобилей одного направления должно приниматься по одному и тому же состоянию поля. Если изменять клетки последовательно непосредственно в исходном массиве, один автомобиль может быть обработан повторно в рамках одного такта. Поэтому сначала формируется логическая маска допустимых

перемещений, после чего все разрешённые перемещения применяются одновременно.

1.3 Двумерная модель дорожного движения BML

Модель BML задаётся квадратной решёткой размером $m \times m$. Состояние клетки s_{ij} принимает одно из трёх значений: 0 — пустая клетка; 1 — автомобиль, движущийся вправо; 2 — автомобиль, движущийся вниз [1]. Один полный такт состоит из двух фаз: сначала одновременно перемещаются автомобили вправо, затем по уже обновлённому полю одновременно перемещаются автомобили вниз. Автомобиль перемещается ровно на одну клетку только при условии, что целевая клетка свободна.

Начальная конфигурация формируется случайно при заданной общей плотности автомобилей. Доля транспортных средств каждого направления может задаваться отдельно. В базовом эксперименте приняты равные потоки: 50 % автомобилей движутся вправо и 50 % — вниз.

1.4 Основные исследуемые показатели

Плотность автомобилей определяется отношением числа занятых клеток N к общему числу клеток поля m^2 :

$$\rho = N / m^2 \quad (1)$$

Средняя скорость в дискретной модели определяется долей автомобилей, которые смогли переместиться за рассматриваемый такт:

$$\langle v(t) \rangle = M(t) / N \quad (2)$$

где $M(t)$ — число переместившихся автомобилей на такте t . Значение $\langle v \rangle = 1$ соответствует движению всех автомобилей, а $\langle v \rangle = 0$ — полной блокировке.

Средний транспортный поток вычисляется как произведение плотности на среднюю скорость:

$$j = \rho \cdot \langle v \rangle \quad (3)$$

Зависимости $\langle v \rangle(\rho)$ и $j(\rho)$ позволяют определить переход от свободного движения к затору и область максимальной пропускной способности системы [1].

2 Постановка задачи и формализация модели

2.1 Принятые допущения

- дорога представлена квадратной клеточной решёткой;
- одна клетка может содержать не более одного автомобиля;
- автомобили имеют одинаковую длину и за такт перемещаются не более чем на одну клетку;
- существуют только два направления движения: вправо и вниз;
- повороты, обгоны, ускорение и торможение отдельно не моделируются;
- границы поля периодические;
- обновление в каждой фазе движения синхронное;
- начальное расположение автомобилей случайное.

2.2 Состояние клеточного поля

Состояние модели хранится в двумерном массиве `field` размером $m \times m$ с типом `int8`. Значения 0, 1 и 2 непосредственно кодируют состояние каждой клетки. Такое представление компактно по памяти и позволяет использовать векторные операции NumPy.

2.3 Правило перехода

На горизонтальной фазе для каждой клетки, содержащей автомобиль типа 1, проверяется соседняя клетка справа. Если она свободна, автомобиль перемещается в неё. Для крайнего правого столбца следующей считается клетка крайнего левого столбца. На вертикальной фазе аналогично обрабатываются автомобили типа 2, но проверяется клетка ниже.

Таким образом, один шаг модели можно записать в виде $F^t \rightarrow F^{t+1/2} \rightarrow F^{t+1}$, где первое преобразование соответствует горизонтальному потоку, а второе — вертикальному.

2.4 Входные и выходные данные

Таблица 1 – Основные параметры модели

Параметр	Обозначение	Смысл	Базовое значение
Размер поля	m	Число клеток по каждой стороне	50
Плотность	ρ	Доля занятых клеток	0,30
Число тактов	T	Продолжительность расчёта	300
Доля потока вправо	α	Доля автомобилей типа 1	0,50
Seed	—	Воспроизводимость начального состояния	1

3 Программная реализация

3.1 Используемые средства

В соответствии с рекомендованным технологическим стеком курса реализация выполнена на Python. Библиотека NumPy используется для работы с массивами и логическими масками, Matplotlib — для построения графиков и анимации. Для определения каталога сохранения изображений применяется `pathlib.Path` [2, 3].

3.2 Назначение функций программы

Таблица 2 – Структура программной реализации

Функция	Назначение	Использование
<code>create_field()</code>	Создаёт случайное начальное состояние поля при заданных m , ρ и α .	Все режимы
<code>bml_step()</code>	Выполняет один полный такт BML: движение вправо и затем вниз.	Все режимы
<code>run_simulation()</code>	Выполняет серию тактов и формирует массив $\langle v(t) \rangle$.	single, experiment
<code>density_experiment()</code>	Перебирает значения ρ , выполняет повторные расчёты и усредняет скорость.	experiment
<code>show_field()</code>	Показывает одно состояние клеточного поля.	single
<code>animate_simulation()</code>	Рассчитывает и отображает анимацию, сохраняет контрольные PNG.	animation

Функция	Назначение	Использование
plot_velocity_time()	Строит график $\langle v \rangle(t)$.	single
plot_velocity_density()	Строит график $\langle v \rangle(\rho)$.	experiment
plot_flow_density()	Строит график $j(\rho)$.	experiment
main()	Хранит пользовательские настройки и выбирает режим работы.	Точка управления

3.3 Инициализация поля

Функция `create_field()` создаёт нулевой массив, определяет количество автомобилей $N = \rho m^2$, после чего с помощью генератора случайных чисел выбирает N различных клеток. Часть выбранных позиций получает состояние 1, остальные — состояние 2. Параметр `seed` позволяет повторно получить ту же начальную конфигурацию.

```
field = np.zeros((size, size), dtype=np.int8)
number_of_cars = int(density * size * size)
positions = rng.choice(size * size, size=number_of_cars, replace=False)
```

3.4 Реализация синхронного движения

Вместо последовательного обхода каждой клетки используется векторная обработка. Операция `np.roll()` циклически сдвигает массив и позволяет получить состояние соседней клетки сразу для всего поля. Логическая операция `&` формирует маску автомобилей, которым разрешено перемещение. После этого

исходные клетки освобождаются, а маска сдвигается в сторону движения и определяет новые позиции.

```
east_cars = field == 1
right_cells_empty = np.roll(field, shift=-1, axis=1) == 0
east_can_move = east_cars & right_cells_empty
```

Такой способ одновременно реализует синхронность обновления и периодические граничные условия. Для вертикального потока используется та же логика со сдвигом по оси 0.

3.5 Режимы работы программы

Пользователь выбирает один из трёх режимов в функции `main()`: `animation` — динамическое отображение эволюции системы; `single` — один расчёт с графиком скорости во времени и конечным состоянием; `experiment` — серия расчётов для разных плотностей с построением зависимостей $\langle v \rangle(\rho)$ и $j(\rho)$.

3.6 Руководство пользователя

Для запуска необходимо установить Python 3 и библиотеки NumPy и Matplotlib. Основные параметры изменяются только в блоке пользовательских настроек функции `main()`. Остальные функции при обычной работе редактировать не требуется.

```
mode = "animation"          # animation / single / experiment
size = 50                   # размер поля 50 x 50
density = 0.30              # плотность автомобилей
steps = 300                 # число тактов
east_fraction = 0.5         # доля машин вправо
seed = 1                    # начальная случайная конфигурация
```

В режиме `animation` параметр `animation_interval` задаёт задержку между кадрами в миллисекундах. Если `save_snapshots = True`, программа сохраняет состояния через `snapshot_step` тактов. Файлы `state_t_0050.png`, `state_t_0100.png` и т. д. сохраняются непосредственно в каталог, где расположен запускаемый .ру-файл.

В режиме `experiment` задаются границы диапазона плотности `density_min` и `density_max`, шаг `density_step`, число независимых запусков `repetitions` и длина

отбрасываемого переходного участка transient. Для корректного усреднения должно выполняться $\text{transient} < \text{steps}$.

4 Вычислительный эксперимент

4.1 Условия эксперимента

Базовый расчёт выполнен на поле 50×50 клеток при равном распределении потоков $\alpha = 0,5$. Для исследования зависимости от плотности использованы значения ρ от 0,05 до 0,95 с шагом 0,05. Для каждой плотности выполнено пять независимых реализаций с $\text{seed} = 0, 1, 2, 3, 4$. Продолжительность каждой реализации составила 300 тактов, первые 200 тактов исключались из усреднения как переходный процесс.

4.2 Эволюция клеточного поля

На рисунке 1 представлены характерные состояния системы при $\rho = 0,30$ и $\text{seed} = 1$. Уже на первых тактах локальные взаимодействия потоков перестраивают случайную исходную конфигурацию. К поздним тактам формируются организованные диагональные структуры, при этом большая часть автомобилей сохраняет возможность движения.

Эволюция BML-модели при $\rho = 0,30$

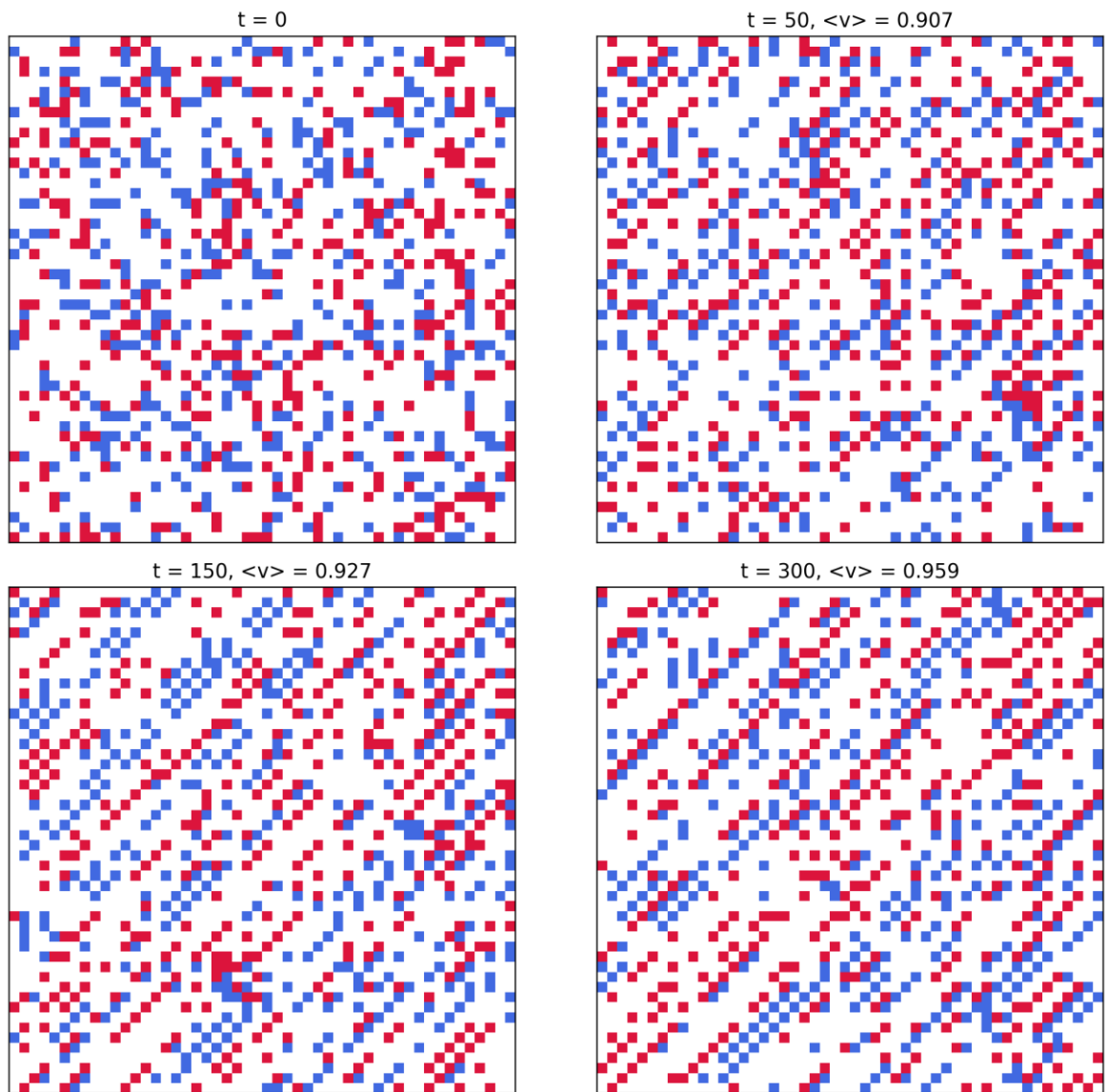


Рисунок 1 – Эволюция BML-модели при $\rho = 0,30$

4.3 Изменение средней скорости во времени

Для $\rho = 0,30$ начальная случайная конфигурация создаёт заметный переходный процесс. Затем средняя скорость возрастает и стабилизируется около 0,96. Это подтверждает целесообразность исключения первых тактов при сравнении установившихся режимов.



Рисунок 2 – Изменение средней скорости при $\rho = 0,30$

4.4 Зависимость средней скорости от плотности

При малых плотностях практически все автомобили перемещаются на каждом такте: при $\rho = 0,05\text{--}0,15$ средняя скорость близка к единице. До $\rho \approx 0,30$ движение остаётся преимущественно свободным. В области $\rho \approx 0,35\text{--}0,50$ наблюдается резкое снижение скорости и высокая чувствительность к начальному расположению автомобилей. При дальнейшем увеличении плотности формируется глобальная блокировка.

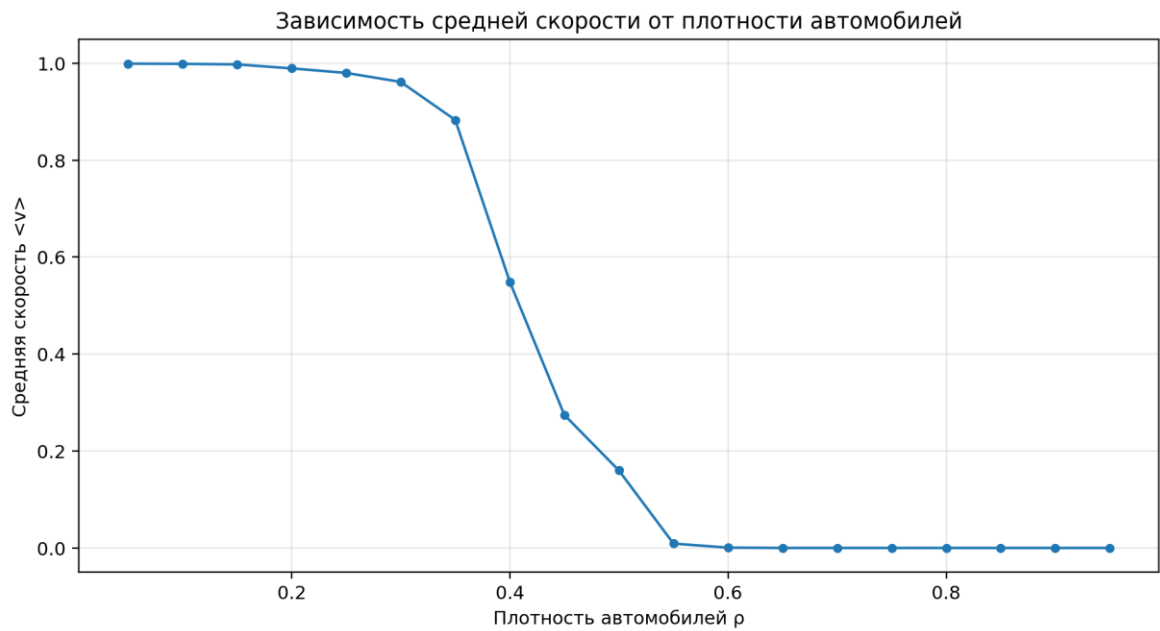


Рисунок 3 – Зависимость средней скорости от плотности автомобилей

4.5 Транспортный поток

На малых плотностях транспортный поток j возрастает почти пропорционально ρ , поскольку $\langle v \rangle \approx 1$. Максимум в выполненном эксперименте получен около $\rho = 0,35$: $j \approx 0,309$. После этого увеличение числа автомобилей приводит не к росту пропускной способности, а к резкому снижению скорости и, как следствие, потока.

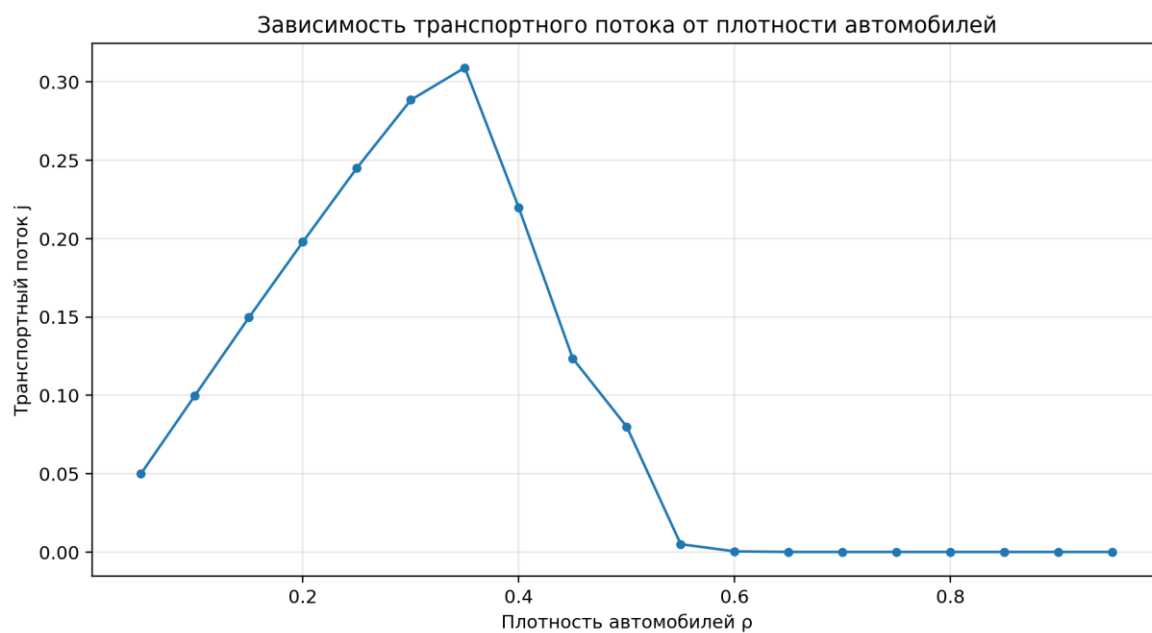


Рисунок 4 – Зависимость транспортного потока от плотности автомобилей

4.6 Численные результаты

Таблица 3 – Усреднённые результаты серии вычислительных экспериментов

ρ	$\langle v \rangle$	$\sigma(\langle v \rangle)$	j
0,05	0,999	0,001	0,050
0,10	0,999	0,002	0,100
0,15	0,998	0,003	0,150
0,20	0,990	0,004	0,198
0,25	0,980	0,005	0,245
0,30	0,962	0,002	0,288
0,35	0,883	0,018	0,309
0,40	0,549	0,193	0,220
0,45	0,274	0,241	0,123
0,50	0,160	0,183	0,080
0,55	0,009	0,020	0,005
0,60	0,001	0,001	0,000
0,65	0,000	0,000	0,000
0,70	0,000	0,000	0,000
0,75	0,000	0,000	0,000
0,80	0,000	0,000	0,000
0,85	0,000	0,000	0,000
0,90	0,000	0,000	0,000
0,95	0,000	0,000	0,000

Рост стандартного отклонения в области $\rho = 0,40\text{--}0,50$ показывает, что в переходной зоне итоговый режим особенно чувствителен к случайной начальной конфигурации. Это согласуется с особенностью BML-модели, указанной в учебных материалах: результат зависит не только от плотности, но и от начального расположения автомобилей [1].

5 Верификация и обсуждение результатов

5.1 Верификация программной реализации

Для проверки корректности реализации использованы инварианты и предельные случаи модели. Во-первых, число автомобилей сохраняется на всех тактах: алгоритм только переносит значения 1 и 2 между клетками, не создавая и не удаляя транспортные средства. Во-вторых, средняя скорость всегда находится в диапазоне $0 \leq \langle v \rangle \leq 1$. В-третьих, при малой плотности скорость стремится к единице, поскольку взаимные блокировки редки. При очень высокой плотности свободных клеток недостаточно, и система приходит к полной блокировке.

Периодические граничные условия проверяются самой операцией `pr.roll()`: сосед крайней правой клетки берётся из крайнего левого столбца, а сосед нижней клетки — из верхней строки. Синхронность обеспечивается вычислением логической маски разрешённых перемещений до изменения поля.

5.2 Интерпретация перехода к затору

Полученная зависимость $\langle v \rangle(\rho)$ демонстрирует качественный фазовый переход. На малых плотностях локальные правила почти не ограничивают движение. При росте ρ возрастает вероятность взаимной блокировки горизонтального и вертикального потоков. Локальные препятствия объединяются в протяжённые структуры, способные остановить значительную часть автомобилей. В выбранной постановке резкий переход начинается примерно в области $\rho = 0,35\text{--}0,40$.

Полную блокировку во всех пяти реализациях данного эксперимента наблюдали при $\rho \geq 0,65$. Это число не следует считать универсальной критической плотностью модели: положение переходной области зависит от размера поля, случайной начальной конфигурации, длительности расчёта и

способа усреднения. Корректнее говорить о критической области для конкретной вычислительной постановки.

5.3 Ограничения модели

- реальное дорожное движение сведено к двум направлениям и единичной скорости;
- не учитываются индивидуальные параметры водителей и транспортных средств;
- отсутствуют повороты, полосность, светофоры и геометрия реальной уличной сети;
- периодические границы соответствуют замкнутой системе без въезда и выезда автомобилей;
- плотность и положение затора зависят от конечного размера поля и начального случайного состояния.

Несмотря на эти ограничения, модель хорошо демонстрирует основную идею клеточных автоматов: сложное глобальное поведение и пробка возникают только из локального правила «двигаться, если следующая клетка свободна», без аварий или централизованного управления.

ЗАКЛЮЧЕНИЕ

В ходе лабораторной работы разработана программная реализация двумерной клеточной модели дорожного движения BML. Сформализованы состояния клеток, периодические граничные условия и двухфазное синхронное правило перехода. Реализация на NumPy использует логические маски и циклические сдвиги массивов, что позволяет компактно и корректно обрабатывать все клетки одновременно.

Разработанная программа поддерживает три режима: анимацию эволюции поля, одиночный расчёт с анализом скорости во времени и серию вычислительных экспериментов по плотности. Реализовано автоматическое сохранение контрольных состояний через заданное число тактов.

Вычислительный эксперимент показал, что при малой плотности автомобилей средняя скорость близка к единице. При увеличении плотности возникает переходная область, в которой скорость резко уменьшается и становится чувствительной к начальной конфигурации. Максимальный транспортный поток для выбранных параметров получен около $\rho = 0,35$, после чего рост плотности снижает пропускную способность. При достаточно высокой плотности формируется полный затор.

Таким образом, поставленная цель достигнута: создана и исследована двумерная клеточная модель дорожного движения, позволяющая наблюдать возникновение заторов и количественно оценивать влияние плотности на среднюю скорость и транспортный поток.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Клеточные автоматы: лекция 3: учебные материалы по курсу «Компьютерное моделирование». – 22 с.
2. Федулов В. А. Компьютерное моделирование. Лекция № 2. Краткие сведения о языке программирования Python: учебные материалы. – М. : МГТУ им. Н. Э. Баумана, кафедра СМ6, 2026. – 52 с.
3. О выполнении заданий по курсу: методические указания. – 5 с.

ПРИЛОЖЕНИЕ А

Листинг ключевых функций модели

Ниже приведены ключевые функции, непосредственно реализующие создание клеточного поля и правило перехода BML. Полная версия программы размещается в репозитории, ссылка на который указывается в начале отчёта.

```
def create_field(size, density, east_fraction, seed=None):
    rng = np.random.default_rng(seed)
    field = np.zeros((size, size), dtype=np.int8)

    number_of_cells = size * size
    number_of_cars = int(density * number_of_cells)
    number_east = int(number_of_cars * east_fraction)

    positions = rng.choice(
        number_of_cells,
        size=number_of_cars,
        replace=False
    )

    east_positions = positions[:number_east]
    south_positions = positions[number_east:]

    east_rows, east_cols = np.unravel_index(
        east_positions, (size, size)
    )
    south_rows, south_cols = np.unravel_index(
        south_positions, (size, size)
    )

    field[east_rows, east_cols] = 1
    field[south_rows, south_cols] = 2

    return field

def bml_step(field):
    # Фаза 1: движение вправо
    east_cars = field == 1
    right_cells_empty = (
        np.roll(field, shift=-1, axis=1) == 0
    )
    east_can_move = east_cars & right_cells_empty
    moved_east = int(np.sum(east_can_move))

    field_after_east = field.copy()
    field_after_east[east_can_move] = 0
    east_new_positions = np.roll(
        east_can_move, shift=1, axis=1
    )
    field_after_east[east_new_positions] = 1

    # Фаза 2: движение вниз
    south_cars = field_after_east == 2
```

```

lower_cells_empty = (
    np.roll(field_after_east, shift=-1, axis=0) == 0
)
south_can_move = south_cars & lower_cells_empty
moved_south = int(np.sum(south_can_move))

new_field = field_after_east.copy()
new_field[south_can_move] = 0
south_new_positions = np.roll(
    south_can_move, shift=1, axis=0
)
new_field[south_new_positions] = 2

moved = moved_east + moved_south
return new_field, moved

```